

**VI THEORY LEARNING OUTCOMES AND ALIGNED COURSE CONTENT**

Module Number	Mapped Course Outcome	Major Learning Outcomes	Theory Content	No. of Hours
1	CO1, CO3	1. Apply and implement self-balancing and multiway tree structures to ensure efficient data organization and access. 2. Perform heap operations and topological sorting to model priority queues and process dependencies in directed graphs. 3. Analyze algorithm efficiency using time and space complexity metrics, including best, worst, and average cases with asymptotic notations.	Advanced Data Structures and Analysis of Algorithms 1.1 Advanced Data Structures: AVL Trees: Implement insertion and deletion operations while maintaining balance using rotations. 1.2 Multiway Trees:B,B++,Red and Black and Splay Trees: Insertion and Deletion Operations. 1.3 Heaps: Execute heap-up and heap-down operations to maintain heap properties (min/max). 1.4 Topological Sort: Perform ordering of DAG nodes using Depth First Search (DFS) 1.5 Algorithm Analysis Fundamentals: Time and Space Complexity: Analyze algorithm efficiency in terms of resource usage. 1.6 Asymptotic Notations: Use Big-O (upper bound), Omega (lower bound), and Theta (tight bound). 1.7 Case-Based Efficiency: Distinguish best, worst, and average case performance scenarios. 1.8 Analysis of non-recursive and recursive algorithms, Recurrences: The substitution method, Recursive tree method, Masters method.	12

(Continued on next page)



Module Number	Mapped Course Outcome	Major Learning Outcomes	Theory Content	No. of Hours
2	CO2	1. Analyze divide-and-conquer strategies using algorithms like Quick Sort, Merge Sort, and Binary Search. 2. Implement and analyze greedy algorithms such as Job Sequencing and fractional knapsack for real-world optimization problems. 3. Apply recursive techniques to solve classical problems like Tower of Hanoi and Min-Max game trees.	Divide and Conquer & Greedy Algorithms 2.1 Understand the concept and strategy of divide and conquer through the Tower of Hanoi case study. 2.2 Implement recursive solutions for Quick Sort, Merge Sort, and Binary Search. 2.3 Min-max Algorithm. 2.4 Greedy Algorithms: Fractional Knapsack Problem. 2.5 Job sequencing with deadlines. 2.6 Huffman coding. 2.7 An Activity Selection Problem. 2.8 Optimal Merge Pattern. 2.9 Analysis of all these algorithms and problem solving.	11
3	CO2	1. Differentiate between bottom-up and top-down approaches in dynamic programming. 2. Formulate the coin change problem using both recursive and dynamic approaches. 3. Comprehend the difference between 0/1 and fractional knapsack problems. 4. Calculate minimal cost tour covering all cities once and returning to the start. 5. Construct a DP table to minimize multiplication operations. 6. Compare OBST with standard binary search trees in terms of efficiency.	Dynamic Programming 3.1 Introduction to Dynamic Programming: Principle of optimality, Bottom-up vs. top-down approaches. 3.2 Case Study: Coin Change Problem. 3.3 Floyd-Warshall Algorithm. 3.4 0/1 Knapsack Problem. 3.5 Traveling Salesman Problem (TSP). 3.6 Matrix Chain Multiplication. 3.7 Optimal Binary Search Tree (OBST). 3.8 Analysis of all these algorithms and problem solving.	12

(Continued on next page)



Mahavir Education Trust's

Shah & Anchor Kutchhi Engineering College

An Autonomous Institute Affiliated to University of Mumbai

Module Number	Mapped Course Outcome	Major Learning Outcomes	Theory Content	No. of Hours
4	CO4	1. Implement the Rabin-Karp algorithm and handle hash collisions. 2. Implement Knuth-Morris-Pratt (KMP) Algorithm and compare its efficiency with previous methods. 3. Formulate the LCS problem using dynamic programming principles.	Pattern Matching Algorithms 4.1 Introduction to Pattern Matching. 4.2 Naïve String-Matching Algorithm. 4.3 Rabin-Karp Algorithm. 4.4 Boyer-Moore Algorithm. 4.5 Knuth-Morris-Pratt (KMP) Algorithm. 4.6 Longest Common Subsequence (LCS). 4.7 Longest Palindromic Subsequence. 4.8 Analysis of all these algorithms and problem solving.	10